

Hiver SDE Intern — Take-Home Assignment

AmazonHelp AI Support Agent

Executive Summary

This project builds a conservative, brand-specific AI customer-support agent using the Customer Support on Twitter dataset. The system classifies incoming messages, retrieves historically similar AmazonHelp support cases, drafts an evidence-grounded reply, and decides whether the case should be auto-handled or escalated.

The project deliberately treats **proof over the system** as the primary objective. A frozen 213-example human-labelled golden set, two conventional baselines, a semantic prototype baseline, LLM classification results, retrieval diagnostics, an LLM-as-judge rubric, and a separate human audit are used to measure where the system succeeds and where it should not be trusted.

Headline results

Measure	Result
Golden evaluation set	213 human-labelled examples
Majority baseline	17.37% accuracy / 0.0185 macro F1
TF-IDF + Logistic Regression	35.21% accuracy / 0.3344 macro F1
Semantic prototype classifier	42.72% accuracy / 0.4306 macro F1
LLM intent classifier	≈61.0% accuracy / ≈0.61 macro F1
Routing experiment	36.2% automation coverage / 70.1% safe precision
LLM judge diagnostic	4.92 / 5 overall on 39 cases
Human audit	1/15 acceptable under strict criteria

Interpretation. The 36.2% automation number is not a production automation-rate estimate. The golden set was intentionally constructed to cover operational intents, including rare categories, rather than sampled from the natural traffic distribution. The human audit also shows that the LLM judge can substantially overestimate response quality.

Scope and non-goals

- The system does not access live orders, accounts, payments, shipment state, or Amazon internal tools.
- It does not claim that a refund, cancellation, replacement, or account action has actually been performed.
- It is a prototype for evidence-grounded support drafting and routing, not a production autonomous agent.

What good means for this brand

For AmazonHelp, useful automation means correctly identifying the customer's operational state, finding comparable historical support behavior, drafting a response without inventing actions or policy, and escalating when evidence or confidence is insufficient.

1. Data, Brand Selection, and Problem Framing

Dataset. The primary source is the Customer Support on Twitter dataset, containing approximately 2.8M tweets across dozens of brands. It contains tweet IDs, authors, inbound/outbound direction, text, timestamps, response IDs, and parent IDs.

Brand selection. AmazonHelp was selected after comparing support corpus size and conversation depth. It contains 169,840 support tweets, 71,049 unique customers, approximately 2.38 support replies per customer, and 7,102 customers receiving five or more replies. Approximately 99.67% of AmazonHelp support tweets have a parent pointer.

This combination makes AmazonHelp suitable for learning from historical support behavior rather than generating generic answers.

Conversation reconstruction. Parent pointers (``in_response_to_tweet_id``) were used as the primary mechanism for reconstruction. This avoids relying on comma-separated response lists as the main source of conversation structure. A terminal support tweet was not treated as a resolved case: the absence of a subsequent recorded response does not prove resolution.

Cleaning. Customer text was normalized by removing Twitter handles and URLs, decoding ``&``, and normalizing whitespace. A blanket 15-character cutoff was intentionally rejected because short multilingual messages can still be meaningful.

Intent discovery. TF-IDF analysis was followed by English and multilingual sentence embeddings. English clustering was partly language-driven, motivating use of ``paraphrase-multilingual-MiniLM-L12-v2``. K-Means was used only to discover themes. Low silhouette values (approximately 0.035–0.047) indicated that clustering should not be treated as ground truth.

Operational taxonomy. The final taxonomy contains 15 supported intents plus OTHER: DELIVERY_DELAY, DELIVERY_NOT_RECEIVED, DELIVERY_TRACKING, DELIVERY_DRIVER_ISSUE, ORDER_STATUS, ORDER_CANCELLATION, MISSING_ITEM, WRONG_ITEM_RECEIVED, RETURN_REPLACEMENT, REFUND_STATUS, ACCOUNT_ACCESS, ACCOUNT_SECURITY, UNEXPECTED_CHARGE, CASHBACK, PRODUCT_ISSUE, and OTHER.

Boundary examples. A missed promised date is DELIVERY_DELAY; a package simply absent is DELIVERY_NOT_RECEIVED; tracking uncertainty is DELIVERY_TRACKING; explicit carrier/driver issues are DELIVERY_DRIVER_ISSUE. A missing item after package arrival remains MISSING_ITEM. A broken product with an explicit return/replace request is RETURN_REPLACEMENT; otherwise it is PRODUCT_ISSUE.

Why these boundaries matter. The hardest errors are not broad domain errors such as delivery vs account. They are neighboring operational states sharing the same vocabulary. The taxonomy therefore reflects support workflow distinctions rather than purely semantic topics.

2. Evaluation Design and Results

Golden set. A frozen set of 213 human-labelled examples was constructed from a 10,000-message candidate sample. Semantic retrieval was used to find candidate examples for each proposed intent, supplemented by random examples. Human annotation remained the source of truth; prototype labels were never treated as ground truth.

Distribution. OTHER=37, ORDER_CANCELLATION=19, REFUND_STATUS=17, DELIVERY_NOT_RECEIVED=16, DELIVERY_TRACKING=16, ACCOUNT_ACCESS=15, PRODUCT_ISSUE=13, ACCOUNT_SECURITY=12, ORDER_STATUS=12, DELIVERY_DELAY=11, UNEXPECTED_CHARGE=9, RETURN_REPLACEMENT=9, CASHBACK=8, DELIVERY_DRIVER_ISSUE=8, MISSING_ITEM=6, WRONG_ITEM_RECEIVED=5.

Baseline comparison.

Model / baseline	Accuracy	Macro F1	Weighted F1
Majority: always OTHER	17.37%	0.0185	0.0514
TF-IDF + Logistic Regression	35.21%	0.3344	0.3437
Semantic prototypes	42.72%	0.4306	0.4003
LLM classifier	≈61.0%	≈0.61	≈0.61

The semantic prototype approach improved over TF-IDF, but remained weak at fine-grained delivery distinctions and rejection. The LLM classifier improved substantially overall, reaching approximately 61% accuracy and 0.61 macro F1, but its remaining errors are concentrated exactly where autonomous support is risky: delivery-delay vs tracking/status, product issue vs return/replacement, refund/cashback, and OTHER.

Retrieval evaluation. Historical pairs were embedded with the multilingual MiniLM model. On the golden set, top-1 similarity averaged 0.897 (median 0.888); the mean top-5 similarity was approximately 0.832. These numbers show that the retriever can find highly similar support language, but similarity alone cannot establish the correct operational state.

An attempted silver-intent retrieval signal was rejected: even unanimous silver-intent agreement matched the human intent only about 56.3% of the time. This was a useful negative result because it prevented a noisy pseudo-labeling mechanism from becoming a hard routing dependency.

Routing. A conservative evaluation policy combining intent, historical similarity, and retrieval agreement yielded 77 AUTO_HANDLE and 136 ESCALATE cases: 36.2% automation coverage with 70.1% safe automation precision. The purpose of this thresholding is not maximum coverage; it is to avoid autonomous responses when evidence is weak or risk is high.

3. Grounded Response Generation and Judge Validation

Generation. The response generator uses retrieved customer/support pairs as evidence. The prompt explicitly prohibits invented policies, refunds, guarantees, completed actions, historical URLs, Twitter handles, tweet IDs, agent identifiers, and dataset artifacts. If evidence is insufficient, the model is instructed to request appropriate information rather than inventing an answer.

A deterministic sanitizer is applied after generation to remove URLs, handles, trailing agent-style signatures, unresolved placeholders, and whitespace/punctuation artifacts. This defense-in-depth approach was added because prompt-only controls were not sufficient.

LLM-as-judge rubric. Each response is scored 1–5 on relevance, groundedness, no hallucination, customer appropriateness, resolution alignment, and overall quality.

Criterion	Mean score (39-case diagnostic sample)
Relevance	4.95 / 5
Groundedness	4.85 / 5
No hallucination	4.97 / 5
Customer appropriateness	4.87 / 5
Resolution alignment	4.92 / 5
Overall	4.92 / 5

Human-vs-judge evidence. The judge result is intentionally not presented as proof. It was generated using the same provider/model family as the response generator, so it is not independent. More importantly, a separate 15-case human audit found only 1/15 responses acceptable under strict human criteria, while the LLM judge rated all 15 as acceptable/high-quality.

The audit identified concrete issues that the automated judge missed, including historical URLs, Twitter handles, unresolved placeholders such as `\_\_email\_\_`, agent-style identifiers, and unsupported claims such as saying a full refund had been processed when the customer message did not establish that the system had actually taken that action.

Representative audit cases.

- A customer asking about a refund received a response containing a historical support URL. The judge still gave the response an overall score of 5.
- A locked-account response contained an unresolved `\_\_email\_\_` placeholder while receiving an overall judge score of 5.
- A 'Full refund! Thank you' acknowledgement generated the unsupported claim that the system had processed the full refund, again receiving a judge score of 5.
- A cashback case produced a reasonable workflow-oriented response, but also included historical Twitter artifacts/URL in the pre-sanitization output.

These examples validate the decision to use the judge only diagnostically and to maintain deterministic artifact checks plus human review.

4. Failure Analysis and Risk-Aware Design

Failure mode 1 — Fine-grained delivery confusion. DELIVERY_DELAY, DELIVERY_NOT_RECEIVED, DELIVERY_TRACKING, and DELIVERY_DRIVER_ISSUE share terms such as package, delivery, tracking, and late. The hypothesis is that semantic representations correctly identify the broad topic but compress operational state distinctions. Mitigation: explicit intent definitions, boundary rules, and conservative escalation.

Failure mode 2 — Refund vs cashback ambiguity. Both categories contain financial-status language such as refund, received, pending, and missing. The hypothesis is lexical/semantic overlap combined with insufficient transaction context. Mitigation: treat ambiguous financial workflows conservatively and escalate when the transaction type is unclear.

Failure mode 3 — OTHER rejection weakness. Generic or casual messages containing Amazon/support vocabulary can be forced into a supported class. This is a classic open-set classification problem. Mitigation: preserve OTHER as an explicit class, add conversational routing for greetings/acknowledgements, and escalate unsupported requests.

Failure mode 4 — Underlying issue vs requested resolution. Customers can describe multiple issues in one message: product damage, missing item, cancellation, and refund. The system must decide whether the intent is the underlying operational problem or the desired resolution. Mitigation: prioritize the operational issue unless there is an explicit return/replacement request.

Failure mode 5 — Grounded-generation artifact leakage. Historical Twitter support contains URLs, handles, signatures, and dataset-specific identifiers. A generator can copy these artifacts even when the semantic response is reasonable. Mitigation: strict generation instructions plus deterministic sanitization.

Risk-aware routing. Account-security and unexpected-charge issues are always escalated. OTHER is escalated. Weak historical similarity triggers escalation. Several historically difficult intents are treated conservatively in the deployed UI. The principle is asymmetric loss: a wrong autonomous support response can be more damaging than an unnecessary human escalation.

Why the routing policy is intentionally conservative. The system does not have live operational state. Historical evidence can suggest what AmazonHelp tended to do, but cannot prove what should happen to a particular customer's current order. Therefore, the agent drafts and routes rather than pretending to execute business operations.

Decision log — non-obvious choices

- AmazonHelp selected for corpus size and conversation depth.
- Parent pointers preferred for conversation reconstruction.
- Terminal support tweets not treated as resolved.
- Handles/URLs removed from customer retrieval text.
- Short multilingual messages retained.
- Multilingual embeddings adopted after language-driven clustering.
- Clustering used for discovery only.
- 15 operational intents + OTHER chosen.
- Human labels treated as golden truth.
- Golden set frozen before final evaluation.
- Silver labels not used as a hard retrieval filter.
- Silver agreement rejected as an escalation signal.
- High-risk intents always escalated.
- Deterministic sanitization added after prompt-only leakage.
- LLM judge treated as diagnostic due human disagreement.

5. What Is Misleading About My Headline Number?

The most tempting headline is **36.2% automation coverage** or **70.1% safe automation precision**. Neither number should be interpreted as production-ready automation capability.

First, the golden set contains 213 human-labelled examples and was deliberately constructed to cover supported intents, including rare categories. It therefore does not represent the natural distribution of incoming support traffic. A production traffic distribution could contain very different proportions of common and rare cases, changing both coverage and precision.

Second, the routing number is tied to a particular evaluation policy and thresholds. It demonstrates a measurable trade-off between automation and escalation; it does not establish that the same threshold will work under distribution shift.

Third, the LLM judge's 4.92/5 result is based on only 39 cases and is not independent of the generation model. The 15-case human audit produced a much more conservative view, accepting only one response under strict criteria. The disagreement is itself an evaluation result: an LLM judge can miss artifacts and unsupported claims that matter to a human reviewer.

Therefore, the correct interpretation is that this prototype demonstrates an end-to-end architecture and evaluation methodology—not a guaranteed autonomous resolution rate.

One more week: highest-value next steps

- **Confidence calibration:** replace fixed thresholds with a held-out calibration set and evaluate selective prediction / risk-coverage curves.
- **Context-aware classification:** use conversation history explicitly for ambiguous follow-ups.
- **Retrieval reranking:** add a cross-encoder or lightweight reranker to separate semantically close operational states.
- **Joint retrieval:** combine query similarity with predicted intent rather than relying on similarity alone.
- **Larger human audit:** evaluate groundedness, unsupported claims, and artifact leakage with a substantially larger sample.
- **Risk-weighted automation:** optimize expected value of automation versus the cost of an incorrect autonomous response.
- **Observability:** track intent distribution, retrieval confidence, escalation rate, latency, and human override rate.

Reproducibility and deployment

The repository separates expensive data preparation from evaluation. Precomputed retrieval artifacts allow the headline deterministic evaluation to be reproduced without re-embedding the full historical corpus. The Streamlit application loads the processed AmazonHelp pairs and embeddings, uses Groq for LLM classification/generation, and maintains conversation state for follow-up questions.

For deployment, large processed artifacts should be stored with Git LFS. API credentials must be supplied through Streamlit Secrets rather than committed to the repository. The raw multi-million-row dataset and local virtual environment are not deployment dependencies.

Final assessment

The strongest part of the project is not the raw classifier score. It is the combination of brand-specific historical grounding, explicit operational intent boundaries, conservative escalation, frozen human evaluation, baseline comparison, judge-vs-human validation, and failure analysis. The system makes uncertainty visible and avoids claiming capabilities that the underlying data cannot support.

Submission note. This report is designed to accompany the repository and README. The README contains the detailed setup, architecture, taxonomy, evaluation methodology, decision log, limitations, and deployment instructions.